

module-05-dictionaries

Chapter 5: Dictionaries and Structuring Data | 37 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch5: dicts, keys/values/items, get/setdefault, pretty printing, nested structures

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

A dictionary maps KEYS to VALUES: `myCat = {'size':'fat','color':'gray'}`. Unlike lists, order does not matter for lookup - `myCat['size']` jumps straight to 'fat'. `keys()/values()/items()` expose the contents; `get(k, default)` avoids `KeyError`; `setdefault(k, default)` inserts only if missing. Dicts nest inside lists and vice versa - that is how real data (guests, inventory, game boards) is structured.

Key ideas to carry forward

- `myCat['size']` -> 'fat'; 'color' in `myCat` checks keys.
- for `k, v` in `myCat.items()`: walks pairs.
- `picnic.get('cups', 0)` returns 0 instead of crashing when the key is absent.
- `pprint.pprint()` prints nested data readably.
- Tic-tac-toe board, guest-counting, fantasy inventory, and list-to-dict are the applied programs.

Classic beginner mistakes

- ! `KeyError` from `myCat['nope']` - use `in` or `get()`.
- ! Assuming two dicts print in the same order as typed (pre-3.7 not guaranteed).

Study order hint: 01-06 dict-vs-list thinking, 07-15 access patterns, 16-20 counting/pretty print, 21-37 board + inventory projects.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. List vs dict - when is a dict better?

A: When lookups are by meaningful key, not position: `myCat['color']` vs remembering index 2. Dicts give $O(1)$ key lookup and self-describing data.

Q2. keys/values/items - what do they give?

A: Views of keys, values, and (key, value) pairs. `for k, v in d.items()` walks both together.

Q3. How do you avoid KeyError?

A: `'k' in d` check, or `d.get(k, default)`. `setdefault(k, default)` inserts the default only when missing.

Q4. What makes a valid dict key?

A: Hashable/immutable values: strings, numbers, tuples of immutables. Lists and dicts cannot be keys.

Q5. How do you model nested real data?

A: Dicts inside lists inside dicts (guests -> dishes). `pprint` shows the shape; count with nested loops.

Q6. Pretty-printing - why bother?

A: `pprint.pprint()` renders nested structures readably for debugging instead of one giant line.

Teach it to others

The one analogy that sticks

A list is a classroom roll (find pupil #7); a dict is a coatroom of labeled hooks (grab the hook marked 'size'). `get(key, default)` is asking the attendant who hands you a spare coat instead of shrugging.

What your students will get wrong

- x Dicts remember insertion order for logic (never rely on order for correctness).
- x Accessing a missing key gives None (it raises `KeyError` - use `get`).
- x Keys can be lists (unhashable - use tuples).

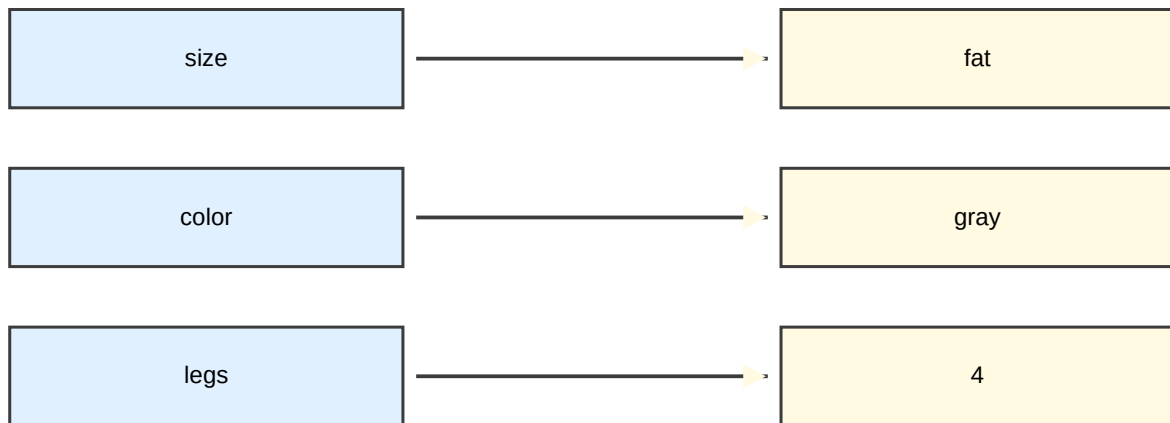
5-minute whiteboard plan

1. Contrast birthday lookup in parallel lists vs one dict.
2. Live-crash on missing key, then rescue with `get/setdefault`.
3. Students model one guest with bring-dishes dict.
4. Exit ticket: count total dishes with a nested loop.

Visualize it - draw these from memory

Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual - keys are labeled hooks, values hang on them



Lookup jumps by label, not position. Missing hook? `get(k, default)` hands a spare coat instead of crashing.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Phone book lookup

```
book = {'al': '555-1', 'bo': '555-2'}
name = 'bo'
print(book.get(name, 'unknown'))
print(book.get('cy', 'unknown'))
# EXPECTED: 555-2 / unknown
```

X2. Word counter

```
text = 'a b a c a b'
counts = {}
for w in text.split():
    counts[w] = counts.get(w, 0) + 1
print(counts)
# EXPECTED: {'a': 3, 'b': 2, 'c': 1}
```

X3. Inventory add

```
inv = {'rope': 1}
loot = ['rope', 'torch', 'rope']
for item in loot:
    inv[item] = inv.get(item, 0) + 1
print(inv)
# EXPECTED: {'rope': 3, 'torch': 1}
```

Program map - all 37 at a glance

```
01 01_dictionary_data_type.py
02 02_dictionary_data_type_spam.py
03 03_dictionaries_vs_lists.py
04 04_dictionaries_vs_lists_spam.py
05 05_dictionaries_vs_lists_birthdays.py
06 06_dictionaries_vs_lists_enter.py
07 07_keys_values_items.py
08 08_keys_values_items_2.py
09 09_keys_values_items_spam.py
10 10_keys_values_items_spam_2.py
11 11_checking_whether_key.py
12 12_get_method.py
13 13_get_method_picnicitems.py
14 14_setdefault_method.py
15 15_setdefault_method_spam.py
16 16_setdefault_method_message.py
17 17_setdefault_method_2.py
18 18_pretty_printing.py
19 19_pretty_printing_a.py
20 20_pretty_printing_pprint.py
21 21_tic_tac_toe.py
22 22_tic_tac_toe_theboard.py
23 23_tic_tac_toe_theboard_2.py
24 24_tic_tac_toe_theboard_3.py
25 25_tic_tac_toe_2.py
26 26_tic_tac_toe_theboard_4.py
27 27_tic_tac_toe_3.py
28 28_tic_tac_toe_4.py
29 29_tic_tac_toe_theboard_5.py
30 30_tic_tac_toe_turn.py
31 31_nested_dictionaries_lists.py
32 32_nested_dictionaries_lists_number.py
33 33_fantasy_game_inventory.py
```

34 34_fantasy_game_inventory_stuff.py
35 35_list_dictionary_function.py
36 36_list_dictionary_function_addtoinvento.py
37 37_list_dictionary_function_inventory.py

Topic: The Dictionary Data Type

01/37 01_dictionary_data_type.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
myCat['size']
```

```
# OUT: 'fat'
```

```
'My cat has ' + myCat['color'] + ' fur.'
```

```
# OUT: 'My cat has gray fur.'
```

02/37 02_dictionary_data_type_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
spam = {12345: 'Luggage Combination', 42: 'The Answer'}
```

Topic: Dictionaries vs. Lists

03/37 03_dictionaries_vs_lists.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> spam = ['cats', 'dogs', 'moose']  
>>> bacon = ['dogs', 'moose', 'cats']  
>>> spam == bacon  
# OUT: False  
  
>>> eggs = {'name': 'Zophie', 'species': 'cat', 'age': '8'}  
>>> ham = {'species': 'cat', 'age': '8', 'name': 'Zophie'}  
>>> eggs == ham  
# OUT: True
```

04/37 04_dictionaries_vs_lists_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = {'name': 'Zophie', 'age': 7}  
spam['color']  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#1>", line 1, in <module>  
# OUT:     spam['color']  
# OUT: KeyError: 'color'
```

05/37 05_dictionaries_vs_lists_birthdays.py [script]

Script: save and run; predict the output first, then compare.

```
birthdays = {'Alice': 'Apr 1', 'Bob': 'Dec 12', 'Carol': 'Mar 4'}

while True:
    print('Enter a name: (blank to quit)')
    name = input()
    if name == '':
        break

    if name in birthdays:
        print(birthdays[name] + ' is the birthday of ' + name)
    else:
        print('I do not have birthday information for ' + name)
        print('What is their birthday?')
        bday = input()
        birthdays[name] = bday
        print('Birthday database updated.')
```

06/37 06_dictionaries_vs_lists_enter.py [script]

Script: save and run; predict the output first, then compare.

```
Enter a name: (blank to quit)
Alice
Apr 1 is the birthday of Alice
Enter a name: (blank to quit)
Eve
I do not have birthday information for Eve
What is their birthday?
Dec 5
Birthday database updated.
Enter a name: (blank to quit)
Eve
Dec 5 is the birthday of Eve
Enter a name: (blank to quit)
```

Topic: The keys(), values(), and items() Methods

07/37 07_keys_values_items.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = {'color': 'red', 'age': 42}  
for v in spam.values():  
# OUT:          print(v)  
  
# OUT: red  
# OUT: 42
```

08/37 08_keys_values_items_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
for k in spam.keys():  
# OUT:          print(k)  
  
# OUT: color  
# OUT: age  
for i in spam.items():  
# OUT:          print(i)  
  
# OUT: ('color', 'red')  
# OUT: ('age', 42)
```

09/37 09_keys_values_items_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = {'color': 'red', 'age': 42}  
spam.keys()  
# OUT: dict_keys(['color', 'age'])  
list(spam.keys())  
# OUT: ['color', 'age']
```

10/37 10_keys_values_items_spam_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = {'color': 'red', 'age': 42}  
for k, v in spam.items():  
# OUT:          print('Key: ' + k + ' Value: ' + str(v))  
  
# OUT: Key: age Value: 42  
# OUT: Key: color Value: red
```

Topic: Checking Whether a Key or Value Exists in a Dictionary

11/37 11_checking_whether_key.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> spam = {'name': 'Zophie', 'age': 7}  
>>> 'name' in spam.keys()  
# OUT: True  
>>> 'Zophie' in spam.values()  
# OUT: True  
>>> 'color' in spam.keys()  
# OUT: False  
>>> 'color' not in spam.keys()  
# OUT: True  
>>> 'color' in spam  
# OUT: False
```

Topic: The get() Method

12/37 12_get_method.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
picnicItems = {'apples': 5, 'cups': 2}  
'I am bringing ' + str(picnicItems.get('cups', 0)) + ' cups.'  
# OUT: 'I am bringing 2 cups.'  
'I am bringing ' + str(picnicItems.get('eggs', 0)) + ' eggs.'  
# OUT: 'I am bringing 0 eggs.'
```

13/37 13_get_method_picnicitems.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
picnicItems = {'apples': 5, 'cups': 2}  
'I am bringing ' + str(picnicItems['eggs']) + ' eggs.'  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#34>", line 1, in <module>  
# OUT:     'I am bringing ' + str(picnicItems['eggs']) + ' eggs.'  
# OUT: KeyError: 'eggs'
```

Topic: The.setdefault() Method

14/37 14_setdefault_method.py [script]

Script: save and run; predict the output first, then compare.

```
spam = {'name': 'Pooka', 'age': 5}
if 'color' not in spam:
    spam['color'] = 'black'
```

15/37 15_setdefault_method_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = {'name': 'Pooka', 'age': 5}  
spam.setdefault('color', 'black')  
# OUT: 'black'  
spam  
# OUT: {'color': 'black', 'age': 5, 'name': 'Pooka'}  
spam.setdefault('color', 'white')  
# OUT: 'black'  
spam  
# OUT: {'color': 'black', 'age': 5, 'name': 'Pooka'}
```

16/37 16_setdefault_method_message.py [script]

Script: save and run; predict the output first, then compare.

```
message = 'It was a bright cold day in April, and the clocks were striking thirteen.'
count = {}

for character in message:
    count.setdefault(character, 0)
    count[character] = count[character] + 1

print(count)
```

17/37 17_setdefault_method_2.py [script]

Script: save and run; predict the output first, then compare.

```
{' ': 13, ',': 1, '.': 1, 'A': 1, 'I': 1, 'a': 4, 'c': 3, 'b': 1, 'e': 5, 'd': 3, 'g': 2, 'i':  
6, 'h': 3, 'k': 2, 'l': 3, 'o': 2, 'n': 4, 'p': 1, 's': 3, 'r': 5, 't': 6, 'w': 2, 'y': 1}
```

Topic: Pretty Printing

18/37 18_pretty_printing.py [script]

Script: save and run; predict the output first, then compare.

```
import pprint
message = 'It was a bright cold day in April, and the clocks were striking
thirteen.'
count = {}

for character in message:
    count.setdefault(character, 0)
    count[character] = count[character] + 1

pprint.pprint(count)
```

19/37 19_pretty_printing_a.py [script]

Script: save and run; predict the output first, then compare.

```
{' ': 13,  
,': 1,  
,': 1,  
'A': 1,  
'I': 1,  
'a': 4,  
'b': 1,  
'c': 3,  
'd': 3,  
'e': 5,  
'g': 2,  
'h': 3,  
'i': 6,  
'k': 2,  
'l': 3,  
'n': 4,  
'o': 2,  
'p': 1,  
'r': 5,  
's': 3,  
't': 6,  
'w': 2,  
'y': 1}
```

20/37 20_pretty_printing_pprint.py [script]

Script: save and run; predict the output first, then compare.

```
pprint.pprint(someDictionaryValue)
print(pprint.pformat(someDictionaryValue))
```

Topic: A Tic-Tac-Toe Board

21/37 21_tic_tac_toe.py [script]

Script: save and run; predict the output first, then compare.

```
theBoard = {'top-L': ' ', 'top-M': ' ', 'top-R': ' ',  
            'mid-L': ' ', 'mid-M': ' ', 'mid-R': ' ',  
            'low-L': ' ', 'low-M': ' ', 'low-R': ' '}
```

22/37 22_tic_tac_toe_theboard.py [script]

Script: save and run; predict the output first, then compare.

```
theBoard = {'top-L': ' ', 'top-M': ' ', 'top-R': ' ',  
            'mid-L': ' ', 'mid-M': 'X', 'mid-R': ' ',  
            'low-L': ' ', 'low-M': ' ', 'low-R': ' '}
```

23/37 23_tic_tac_toe_theboard_2.py [script]

Script: save and run; predict the output first, then compare.

```
theBoard = {'top-L': 'O', 'top-M': 'O', 'top-R': 'O',  
            'mid-L': 'X', 'mid-M': 'X', 'mid-R': ' ',  
            'low-L': ' ', 'low-M': ' ', 'low-R': 'X'}
```

24/37 24_tic_tac_toe_theboard_3.py [script]

Script: save and run; predict the output first, then compare.

```
theBoard = {'top-L': ' ', 'top-M': ' ', 'top-R': ' ',
            'mid-L': ' ', 'mid-M': ' ', 'mid-R': ' ',
            'low-L': ' ', 'low-M': ' ', 'low-R': ' '}

def printBoard(board):
    print(board['top-L'] + '|' + board['top-M'] + '|' + board['top-R'])
    print('-+-+-')
    print(board['mid-L'] + '|' + board['mid-M'] + '|' + board['mid-R'])
    print('-+-+-')
    print(board['low-L'] + '|' + board['low-M'] + '|' + board['low-R'])
printBoard(theBoard)
```

25/37 25_tic_tac_toe_2.py [script]

Script: save and run; predict the output first, then compare.

```
| |  
-+--+  
| |  
-+--+  
| |
```

26/37 26_tic_tac_toe_theboard_4.py [script]

Script: save and run; predict the output first, then compare.

```
theBoard = {'top-L': 'O', 'top-M': 'O', 'top-R': 'O', 'mid-L': 'X', 'mid-M':  
'X', 'mid-R': ' ', 'low-L': ' ', 'low-M': ' ', 'low-R': 'X'}  
  
def printBoard(board):  
    print(board['top-L'] + '|' + board['top-M'] + '|' + board['top-R'])  
    print('-+-+-')  
    print(board['mid-L'] + '|' + board['mid-M'] + '|' + board['mid-R'])  
    print('-+-+-')  
    print(board['low-L'] + '|' + board['low-M'] + '|' + board['low-R'])  
printBoard(theBoard)
```

27/37 27_tic_tac_toe_3.py [script]

Script: save and run; predict the output first, then compare.

```
0|0|0
-+-+
X|X|
-+-+
| |X
```

28/37 28_tic_tac_toe_4.py [script]

Script: save and run; predict the output first, then compare.

```
0|0|0
-+-+
Traceback (most recent call last):
  File "ticTacToe.py", line 10, in <module>
    printBoard(theBoard)
  File "ticTacToe.py", line 6, in printBoard
    print(board['mid-L'] + '|' + board['mid-M'] + '|' + board['mid-R'])
KeyError: 'mid-L'
```

29/37 29_tic_tac_toe_theboard_5.py [script]

Script: save and run; predict the output first, then compare.

```
theBoard = {'top-L': ' ', 'top-M': ' ', 'top-R': ' ', 'mid-L': ' ', 'mid-M': ' ', 'mid-R': ' ', 'low-L': ' ', 'low-M': ' ', 'low-R': ' '}

def printBoard(board):
    print(board['top-L'] + '|' + board['top-M'] + '|' + board['top-R'])
    print('-+-+-')
    print(board['mid-L'] + '|' + board['mid-M'] + '|' + board['mid-R'])
    print('-+-+-')
    print(board['low-L'] + '|' + board['low-M'] + '|' + board['low-R'])
turn = 'X'
for i in range(9):
    printBoard(theBoard)
    print('Turn for ' + turn + '. Move on which space?')
    move = input()
    theBoard[move] = turn
    if turn == 'X':
        turn = 'O'
    else:
        turn = 'X'
printBoard(theBoard)
```

30/37 30_tic_tac_toe_turn.py [script]

Script: save and run; predict the output first, then compare.

```
| |  
-+-+--  
| |  
-+-+--  
| |  
Turn for X. Move on which space?  
mid-M  
| |  
-+-+--  
|X|  
-+-+--  
| |  
Turn for O. Move on which space?  
low-L  
| |  
-+-+--  
|X|  
-+-+--  
O| |  
  
--snip--  
  
O|O|X  
-+-+--  
X|X|O  
-+-+--  
O| |X  
Turn for X. Move on which space?  
low-M  
O|O|X  
-+-+--  
X|X|O  
-+-+--  
O|X|X
```

Topic: Nested Dictionaries and Lists

31/37 31_nested_dictionaries_lists.py [script]

Script: save and run; predict the output first, then compare.

```
allGuests = {'Alice': {'apples': 5, 'pretzels': 12},
             'Bob': {'ham sandwiches': 3, 'apples': 2},
             'Carol': {'cups': 3, 'apple pies': 1}}

def totalBrought(guests, item):
    numBrought = 0
    for k, v in guests.items():
        numBrought = numBrought + v.get(item, 0)
    return numBrought

print('Number of things being brought:')
print(' - Apples      ' + str(totalBrought(allGuests, 'apples')))
print(' - Cups        ' + str(totalBrought(allGuests, 'cups')))
print(' - Cakes        ' + str(totalBrought(allGuests, 'cakes')))
print(' - Ham Sandwiches ' + str(totalBrought(allGuests, 'ham sandwiches')))
print(' - Apple Pies    ' + str(totalBrought(allGuests, 'apple pies')))
```

32/37 32_nested_dictionaries_lists_number.py [script]

Script: save and run; predict the output first, then compare.

Number of things being brought:

- Apples 7
- Cups 3
- Cakes 0
- Ham Sandwiches 3
- Apple Pies 1

Topic: Fantasy Game Inventory

33/37 33_fantasy_game_inventory.py [script]

Script: save and run; predict the output first, then compare.

```
Inventory:
12 arrow
42 gold coin
1 rope
6 torch
1 dagger
Total number of items: 62
```

34/37 34_fantasy_game_inventory_stuff.py [script]

Script: save and run; predict the output first, then compare.

```
# inventory.py
stuff = {'rope': 1, 'torch': 6, 'gold coin': 42, 'dagger': 1, 'arrow': 12}

def displayInventory(inventory):
    print("Inventory:")
    item_total = 0
    for k, v in inventory.items():
        # FILL IN THE CODE HERE
    print("Total number of items: " + str(item_total))

displayInventory(stuff)
```

Topic: List to Dictionary Function for Fantasy Game Inventory

35/37 35_list_dictionary_function.py [script]

Script: save and run; predict the output first, then compare.

```
dragonLoot = ['gold coin', 'dagger', 'gold coin', 'gold coin', 'ruby']
```

36/37 36_list_dictionary_function_addtoinventory.py [script]

Script: save and run; predict the output first, then compare.

```
def addToInventory(inventory, addedItems):  
    # your code goes here  
  
inv = {'gold coin': 42, 'rope': 1}  
dragonLoot = ['gold coin', 'dagger', 'gold coin', 'gold coin', 'ruby']  
inv = addToInventory(inv, dragonLoot)  
displayInventory(inv)
```

37/37 37_list_dictionary_function_inventory.py [script]

Script: save and run; predict the output first, then compare.

Inventory:

45 gold coin

1 rope

1 ruby

1 dagger

Total number of items: 48